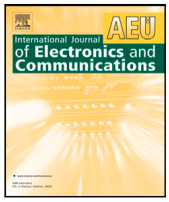
Int. J. Electron. Commun. (AEÜ) 162 (2023) 154575

Contents lists available at ScienceDirect

Int. J. Electron. Commun. (AEÜ)

journal homepage: www.elsevier.com/locate/aeue



Regular paper

An 8-bit integer true periodic orbit PRNG based on delayed Arnold's cat map

Vianney Boniface Ekani Mebenga [a], Venkata Reddy Kopparthi [b], Hermann Djeugoue Nzeuga [c], J.S. Armand Eyebe Fouda [c], Guy Morgan Djeufa Dagoumguei [c], Georges Bell Bitjoka [a], P. Rangababu [d], Samrat L. Sabat [b,*]

[a] ENSPY, University of Yaounde 1, Cameroon
[b] Centre for Advanced Studies in Electronics Science and Technology, School of Physics University of Hyderabad, Hyderabad, India
[c] Département de Physique, Université de Yaoundé I, Yaoundé, Cameroon
[d] Department of Electronics and Communication Engineering, National Institute of Technology Meghalaya, Shillong, India

ARTICLE INFO

Keywords:
Arnold's Cat Map (ACM)
Pseudorandom number generators (PRNG)
Field Programmable Gate Array (FPGA)
NIST statistical test

ABSTRACT

Designing a pseudo-random number generator (PRNG) exhibiting true periodic orbits with large periods is promising for embedded applications. Implementing chaotic behaviors on a finite precision digital platform using conventional unsigned integer or floating-point arithmetic yields short period limit-cycles. It leads the output to diverge from the true chaotic orbits. This paper presents an unsigned integer encoded PRNG based on the delayed quantized Arnold's Cat Map (QACM) that exhibits true periodic orbits with large periods. The proposed PRNG is obtained by delaying one of the 2-D QACM outputs with a linear feedback shift register (LFSR) thereby increasing its period. By investigating its period, we show that the periodicity increases with an increase in the delay and bit-precision. We also present the proposed PRNG unit's hardware architecture and its synthesis results targeting on a Xilinx Zynq 7000 Field Programmable Gate Array (FPGA) with 8-bit precision and delay $\delta = 58$. Further, we synthesize the design using 90 nm CMOS technology for ASIC realization. The synthesis results demonstrate the throughput as 2.208 Gbps and 5.46 Gbps at 138 MHz and 342 MHz operating frequency on FPGA and ASIC respectively. We experimentally evaluate the performance of random numbers using the NIST 800-22-1 tests along with the correlation and key sensitivity tests.

1. Introduction

Modern embedded systems such as the Internet of Things (IoT) require the digital implementation of pseudo-random number generators (PRNG) for data encryption. Chaotic systems are popularly used for generating pseudo-random numbers because they are deterministic, recursive, non-periodic, and extremely sensitive to initial conditions and control parameters. Different chaotic maps such as the Logistic map, Bernoulli shift map, Tent map, and Zigzag map have generated PRNG exhibiting varied degrees of randomness and complexity. However, the digital implementation of such chaotic systems gives periodic output known as pseudo-chaotic orbits with short periods and strong correlations. It is mainly due to the finite precision effect and rounding errors irrespective of fixed or floating-point realization [1]. The influence of finite precision on discrete chaotic maps is investigated in [2]. A method based on a quadratic congruential generator proposed in [3] counteracts the performance degradation of digital chaos and reported a period of 768 under 8-bit precision. The keyspace of the resulting PRNG is approximated as 2^{189} under 32-bit precision; however, the corresponding period was not provided. The modified binary shift chaotic map (M-BSCM) overcomes the degradation of chaotic orbits due to finite precision [1]. The period of the resulting PRNG is approximated to 2^{128} under 32-bit precision. In cryptography, a minimum period length of 2^{128} is required for a pseudo-random sequence to ensure the security of stream ciphers.

Different chaotic maps such as logistic map, Lorenz map, and coupled skew Tent maps have been implemented and verified on Field Programmable Gate Array (FPGA) hardware [4–6]. In literature, the chaotic degradation is partially resolved by either increasing the bit-width of the design or cascading multiple chaotic systems or by using perturbation techniques [1,7]. Each method has its advantages and disadvantages [1]. In addition, low-bit length integer chaotic PRNG is the most promising solution for IoT applications [8]. Indeed, it is impossible to obtain a true chaotic orbit using a finite precision

* Corresponding author.
E-mail addresses: soleilviann@yahoo.fr (V.B.E. Mebenga), kvenkatareddy@uohyd.ac.in (V.R. Kopparthi), mahernzeuga@yahoo.fr (H.D. Nzeuga), efoudajsa@yahoo.fr (J.S.A.E. Fouda), mdjeufa@yahoo.com (G.M.D. Dagoumguei), belltver@yahoo.fr (G.B. Bitjoka), p.rangababu@nitm.ac.in (P. Rangababu), slssp@uohyd.ac.in (S.L. Sabat).

https://doi.org/10.1016/j.aeue.2023.154575
Received 31 December 2022; Accepted 7 February 2023
Available online 15 February 2023

computing system due to rounding errors. Hence, integer arithmetic that avoids the rounding errors in finite precision computing systems is a promising choice to generate pseudo-random numbers (PRN) [1,9]. Although the integer realization can avoid the rounding error issue, it has the disadvantages of short-period limit cycles leading to the periodic characteristic of chaotic maps. Thus, it is interesting to realize the chaotic map using integer arithmetic with a very large true period of the orbits, indistinguishable from the true chaotic orbits [1].

Panda et al. proposed a coupled-variable input linear congruential generator (LCG) that outputs orbit with maximal period and has reduced latency and area consumption compared to the conventional equivalent LCG [10]. Despite the improvement in their architecture, the resource utilization remains as high as the two coupled LCG systems. Recently the true periodic orbits were obtained by replacing the least significant zero bits of the digital realization of the chaotic system with random bits derived from a PRNG [11]. It was later shown that such systems are less sensitive to initial conditions [1]. The Arnold cat map (ACM) is an example of a chaotic system whose digital implementation yields true periodic orbits while preserving its sensitivity to initial conditions. Therefore, it constitutes a good candidate for applying true periodic orbits to chaos-based cryptography [12]. The ACM also has the properties like chaotic, area-preserving, ergodic, mixing, and is invertible [13,14]. The original cat map is generalized and discretized in the phase space for hardware realization to obtain the quantized ACM (QACM). It has been shown that with discretization, the period does not exceed $3m$, m being the modulo value. In the case of n-bit precision ($m = 2^n$, $n \in \mathbb{N}_{\geq 1}$), the period of the QACM is only equal to $T_n = 3 \cdot 2^{n-2}$, $n > 1$, which is effectively small enough to be used as a PRNG. Souza et al. proposed to increase the period by considering $m = 3^s$ and thereafter reducing the output values into n bits ($2^n < 3^s$), with $n, s \in \mathbb{N}_{\geq 1}$ [15]. In [16], the authors suggested coupling two 8-dimensional 8-bit QACM to achieve the dynamics with 10^{27} period [16]. Performing such a high-dimensional system is hardware costly, although the achieved period is reasonably large. Recently, the QACM was perturbed with a finite precision nonlinear term that contributes to extending the keyspace and increasing the period of the QACM [17]. The obtained piece-wise linear cat map (PWLCM) is easy to implement and can achieve a period of 10^{513} for only 10-bit precision. However, it should be emphasized that such a large period corresponds to the least common multiple of small and incommensurable individual periods of the set of initial conditions. For the system to act as a standalone PRNG, the period of each initial condition needs to be increased.

In this paper, we propose a delayed QACM to generate random sequences. One of the outputs of 2-dimensional QACM is delayed by a fixed number of samples with a linear feedback shift register (LFSR). It increases the period of the chaotic system. Indeed, it is known that LFSRs increase the period of a system when used in a feedback loop. Our idea is to use such a strategy to map multiple initial conditions. Hence the delayed QACM will have an increased keyspace and period compared to the conventional QACM by considering a delay inserted in one of its inputs. The corresponding delayed QACM (DQACM) is designed with n-bit unsigned binary integer encoding format instead of fixed or floating-point representations. We investigate the period distribution of the proposed DQACM as a function of the precision n and the delay δ to determine the parameter setting for which the system is passing NIST tests, hence a perfect system for generating pseudo-random numbers. We also propose a hardware architecture and demonstrate the functionality of the DQACM for generating pseudo-random numbers on a Xilinx Zynq 7000 FPGA. Further, we synthesized the design in ASIC with 90 nm CMOS technology using 8-bit unsigned integer-point arithmetic.

The rest of the paper is organized as follows: Section 2 is devoted to the description of the proposed system and its period distribution; Section 3 presents the hardware architecture of the proposed PRNG and synthesis results on FPGA and ASIC implementation; Section 4 presents the security analysis and experimental results; Section 5 compares the proposed architecture with state of the art PRNGs and is followed by a conclusion in Section 6.

Table 1
Symbols used in the Manuscript.

Symbols	Meaning
x_t, y_t	Phase space variables
$\mathbf{x}$	$\mathbf{x} = (x_0, x_1, \ldots, x_\delta)^T$
t	Time index
m	Modulo of the map
n	Number of encoding bits or precision ($m = 2^n$)
T_n	QACM period
δ	Delay time
$A(t)$	$2 \times (\delta + 2)$ dimensional matrix at time index t
$P(z)$	Characteristic polynomial of the DQACM in $\mathbb{F}_2$

2. Proposed delayed QACM: DQACM

Some important symbols used in the manuscript and their corresponding meanings are tabulated in Table 1.

2.1. Brief recall on the QACM

The ACM is a two-dimensional chaotic map defined as

$$\begin{cases} x_{t+1} = & x_t + y_t \\ y_{t+1} = & x_{t+1} + y_t \end{cases} \mod m, \tag{1}$$

where $t \in \mathbb{N}$ is the time index and $m \in \mathbb{N}_{\geq 1}$. For simplification purpose, we consider $m = 2^n$. The case $n = 0$ corresponds to the continuous discrete-time ACM with $(x, y) \in [0, 1)^2$, $m = 1$. The continuous discrete-time ACM is chaotic. The case $n > 0$ corresponds to the quantized ACM (QACM) and obtained with $(x, y) \in [0, m)^2$ and $m \in \mathbb{N}_{>1}$. The QACM is periodic and its period depends on n as

$$T_n = 2^{n-2} \cdot T_2,\ n > 1, \tag{2}$$

with $T_1 = T_2 = 3$.

2.2. Proposed delayed QACM

The QACM preserves the mixing properties of the continuous chaotic ACM, but its short period given in Eq. (2) limits its use as a PRNG. In order to address this drawback, we propose to delay the x-dimension of the 2-D QACM as described in the following equation:

$$\begin{cases} x_{t+1} = & x_{t-\delta} + y_t \\ y_{t+1} = & x_{t+1} + y_t \end{cases} \mod 2^n, \tag{3}$$

where δ is the delay and $n \in \mathbb{N}_{\geq 1}$. Indeed, the QACM outputs a single orbit corresponding to the single input initial condition. Delaying the x-dimension with δ corresponds to considering $\delta + 1$ different initial values of x for a single output orbit. At each step, the system updates the orbit corresponding to $x_{t-\delta}$ to get x_{t+1} (i.e., a delayed sample $x_{t-\delta}$ is used to evaluate the current value x_{t+1}) and wait for $\delta + 1$ iterations before coming back to x_{t+1}. In contrast, the y-dimension is updated at each iteration. Such a mixing of initial conditions or imbrication of trajectories increases the system's period, depending on the delay δ. This approach is equivalent to extending the dimension of the system by increasing the number of initial conditions. However, the system remains 2-dimensional and runs faster as the conventional 2-D QACM, given that only a single couple (x, y) is updated per iteration. The corresponding DQACM can be equated as

$$\begin{pmatrix} x_{t+1} \\ y_{t+1} \end{pmatrix} = A(t) \begin{pmatrix} x_0 \\ \cdots \\ x_\delta \\ y_\delta \end{pmatrix} \mod 2^n, \quad t \geq \delta, \tag{4}$$

where

$$A(t) = \left(a_{i,j}(t)\right), \tag{5}$$

is a $2 \times (\delta + 2)$ matrix, $i \in \{1, 2\}$, $1 \leq j \leq \delta + 2$, with

$$\begin{cases} a_{1,1}(t+1) = a_{1,\delta+1}(t) + a_{1,\delta+2}(t), \\ a_{2,1}(t+1) = a_{2,\delta+1}(t) + a_{2,\delta+2}(t), \\ a_{1,\delta+2}(t+1) = a_{2,1}(t+1), \\ a_{2,\delta+2}(t+1) = a_{2,\delta+1}(t) + 2a_{2,\delta+2}(t), \\ a_{i,j}(t+1) = a_{i,j-1}(t), \quad 1 < j < \delta + 2. \end{cases} \tag{6}$$

The initial matrix elements are such that $a_{1,1}(\delta) = a_{1,\delta+2}(\delta) = a_{2,1}(\delta) = 1$, $a_{2,\delta+2}(\delta) = 2$ and $a_{i,j}(\delta) = 0$ for $1 < j < \delta+2$. The initial condition values of x_t for $0 \leq t \leq \delta$ and y_δ are chosen from the phase space.

2.3. Period distribution of the DQACM in $\mathbb{F}_2$

We investigate the period distribution of the DQACM as a function of δ and the number of encoding bit n. In the case $\delta = 0$, it is well known that the period depends on n as described in Eq. (2) [14]. In order to establish a similar relationship for $\delta > 0$, we first consider the case $n = 1$ that corresponds to the Galois field $\mathbb{F}_2$ and which is the basic one. The matrix elements of the corresponding system are reduced to 0 and 1. Considering Eq. (6), the characteristic polynomial of the corresponding system is [18,19]

$$P(z) = z^{\delta+2} + z^{\delta+1} + 1. \tag{7}$$

The maximal period of the corresponding system is $T_{max} = 2^{\delta+2} - 1$ [18,19]. Such a period is obtained if the polynomial is primitive. A list of primitive polynomials in $\mathbb{F}_2$ has been provided in [20]. We computed the period of the DQACM for $0 \leq \delta \leq 25$. The corresponding results are tabulated in Table 2. It comes from this table that there are only seven primitive polynomials for this range of δ, namely $\delta = 0, 1, 2, 4, 5, 13, 20$. Such a result is confirmed by the list in [20].

2.4. Extension of the DQACM to $\mathbb{F}_{2^n}$

We further investigate the period of the DQACM in the extension field $\mathbb{F}_{2^n}$ for $1 < n \leq 8$ as reported in Table 2. From this table, we observe that for a given delay value δ, the period distribution as a function of n gives a similar relationship as established for the QACM in Eq. (2) [15], hence

$$T_n(\delta) = 2^{n-2} \cdot T_2(\delta), \ n > 1. \tag{8}$$

Furthermore, except for $\delta = 0$ and $\delta = 3$, we can observe that $T_2(\delta) = 2T_1(\delta)$. Thus, the period $T_n(\delta)$ of the system is strongly related to $T_1(\delta)$ that can be considered as the fundamental period. The fundamental period depends on the delay δ and the case $\delta = 0$ corresponds to the well known QACM with $T_1(0) = T_2(0) = 3$, hence $T_n(0) = 3 \cdot 2^{n-2}$. In the case the characteristic polynomial defined in Eq. (7) is primitive, all the initial conditions generate orbits with a minimal period $T_{min}(\delta) = 2^{\delta+2} - 1$, except for the steady state $(\mathbf{x} = 0, y = 0)$ for which $T_n(\delta) = 1$.

3. Hardware architecture

Our purpose is to design a PRNG based on the DQACM. Therefore, parameters δ and n should be chosen such that the period of the corresponding DQACM is large enough for its output to pass NIST tests. For single-bit precission, $n = 1$, the suitable choice of δ for the system to exhibit a single period for all the initial conditions is such that $T_1(\delta) = T_{min}(\delta) = 2^{\delta+2} - 1$. In such a case, only the steady state $(\mathbf{x} = 0, y = 0)$ exhibiting $T_n(\delta) = 1$ (i.e., a period-1) limit-cycle is to be avoided. The other $2^{\delta+2} - 1$ number of initial conditions, chosen from the phase space, exhibits period $T_{min}(\delta)$. For n-bit precision and delay δ, the proposed architecture has $N_{n,\delta} = 2^{n(\delta+2)} - 1$ number of initial conditions that produces PRNG output. Although it has the flexibility to choose the initial conditions from a larger phase space, the periodicity depends on the delay i.e., δ and determining such a delay is not trivial. A list of primitive polynomials provided in [18] shows that Eq. (7) is also primitive for $\delta = 58$ and $\delta = 61$. Choosing a large δ value such that

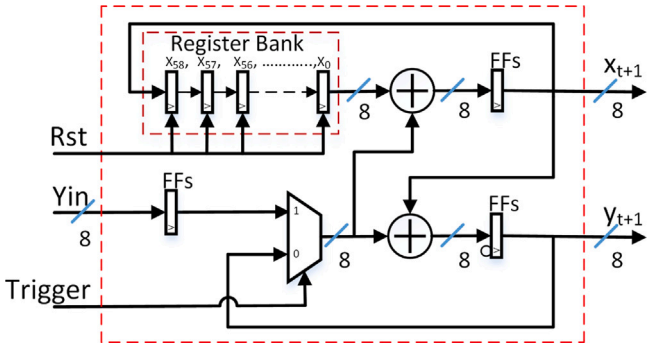


Fig. 1. Hardware architecture of the 8-bit delayed QACM PRNG.

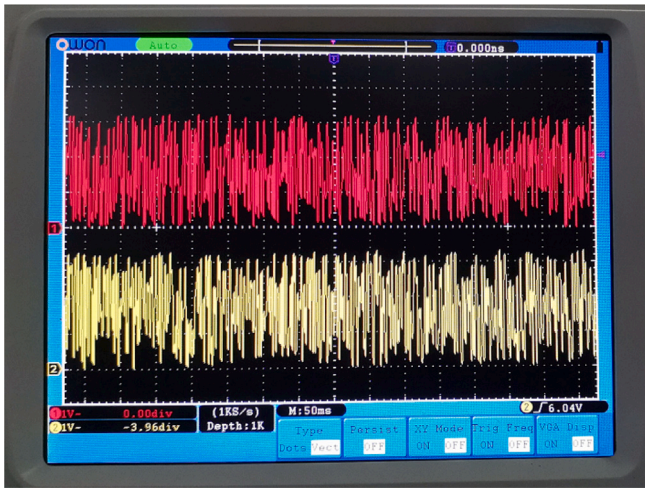


Fig. 2. Experimentally observed pseudo-random number sequences.

$N_{n,\delta}$ is large enough allows us to obtain a PRNG with a large key-space, but is also hardware costly. Thus, we consider $\delta = 58$ and therefore propose to design a PRNG that outputs 8-bit encoded unsigned integers ($n = 8$). The minimal period exhibited by such a PRNG is $T_1(58) = 2^{60} - 1$. Fig. 1 presents the proposed hardware architecture of the two-channel DQACM with δ =58, which produces 8-bit PRNG per channel corresponding to Eq. (3). The *Rst* signal resets the register bank with initial values **x**, whereas Y_{in} corresponds to the initial value y. **x** and y are chosen from the phase space. The initial values are loaded when the *Trigger* command is set to one, and the system starts iterating when *Trigger* is zero. The x channel iterates on the rising edges of the clock input, while the y channel iterates on the falling edges. This approach makes the system run faster and outputs two random samples (x and y) per clock cycle.

Table 3 tabulates the results obtained after synthesizing the design targeting Xilinx Zynq 7000 FPGA. It demonstrates that the design can operate on 138 MHz frequency at a reasonable hardware resources. Fig. 2 presents the experimental PRNG sequences of the proposed architecture when implemented on Xilinx Zynq 7000 FPGA. The architecture gives eight random bits per channel in a clock cycle leading to 2.208 Gbps throughput without using any DSP slice. Similarly, the ASIC synthesis using 90 nm technology library results the throughput of 5.46 Gbps with 0.24 pJ/bit energy consumption.

4. Security analysis

The security analysis of the PRNG is performed by evaluating its randomness under different parameter settings. For randomness analysis, the PRNG sequence obtained from the hardware is tested using a phase

Table 2
Period distribution of the DQACM.

δ/n	1	2	3	4	5	6	7	8
0	3	3	6	12	24	48	96	192
1	7	14	28	56	112	224	448	896
2	15	30	60	120	240	480	960	1920
3	21	42	42	84	168	336	672	1344
4	63	126	252	504	1008	2016	4032	8064
5	127	254	508	1016	2032	4064	8128	16256
6	63	126	252	504	1008	2016	4032	8064
7	73	146	292	584	1168	2336	4672	9344
8	889	1778	3556	7112	14224	28448	56896	113792
9	1533	3066	6132	12264	24528	49056	98112	196224
10	3255	6510	13020	26040	52080	104160	208320	416640
11	7905	15810	31620	63240	126480	252960	505920	1011840
12	11811	23622	47244	94488	188976	377952	755904	1511808
13	32767	65534	131068	262136	524272	1048544	2097088	4194176
14	255	510	1020	2040	4080	8160	16320	32640
15	273	546	1092	2184	4368	8736	17472	34944
16	253921	507842	1015684	2031368	4062736	8125472	16250944	32501888
17	413385	826770	1653540	3307080	6614160	13228320	26456640	52913280
18	761763	1523526	3047052	6094104	12188208	24376416	48752832	97505664
19	5461	10922	21844	43688	87376	174752	349504	699008
20	4194303	8388606	16777212	33554424	67108848	134217696	268435392	536870784
21	2088705	4177410	8354820	16709640	33419280	66838560	133677120	267354240
22	2097151	4194302	8388604	16777208	33554416	67108832	134217664	268435328
23	10961685	21923370	43846740	87693480	175386960	350773920	701547840	1403095680
24	298935	597870	1195740	2391480	4782960	9565920	19131840	38263680
25	125829105	251658210	503316420	1006632840	2013265680	4026531360	8053062720	16106125440

Table 3
Hardware resource utilization of the DQACM architecture.

	FPGA	ASIC
Data format	Unsigned integer	Unsigned integer
Technology	Zynq 7000	TSMC 90 nm
LUT	65	Gate Count
LUTRAM	31	= 2.73 K
FF	290	NAND2X1
BUFG	1	–
Fmax (MHz)	138	342
Throughput (Gbps)	2.208	5.46
Power(mW)	11 (dynamic)	1.3
Energy/bit (pJ/bit)	–	0.24

portrait test, key sensitivity, and correlation test (auto-correlation, cross-correlation). Along with this, National Institute of Standards and Technology (NIST) released software NIST 800-22 is also used to test the binary sequence's randomness. It is popularly used for PRNG testing. We generated 100 sequences of 10^6 bits for the statistical and NIST test.

4.1. Key space analysis

The system contains 60 parameters that are 8-bit encoded each. Except the trivial point ($\mathbf{x} = 0, y = 0$), the number of initial conditions with at least period $2^{60} - 1$ is $2^{480} - 1$ that corresponds to a 480-bit length key-space. In modern cryptography, a key-space length of 256 bits is sufficiently large to resist all kinds of statistical attacks.

4.2. Phase portrait test

The phase space and, subsequently, the randomness of a random number sequence can be analyzed from the first return map or the phase space plot. The phase space plot of sequences generated from our PRNG is shown in Fig. 3. The experimental phase space plot is shown in Fig. 4. We set $\mathbf{x} = (1, 1, \ldots, 1)$ and $y = 1$. From this figure, it is observed that the data is scattered in the plane, hence is random.

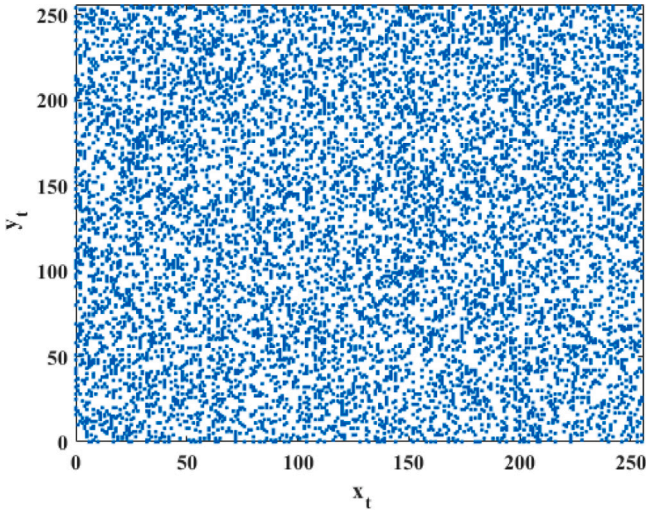


Fig. 3. Phase plot between x_t and y_t.

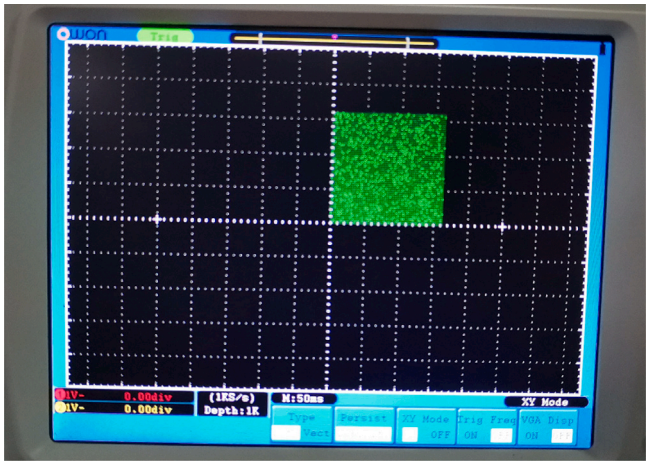


Fig. 4. Experimentally observed phase plot in oscilloscope.

Table 4
Key sensitivity analysis of the PRNG on FPGA.

(B1, B2)	(X3, X4)	(X3, Y3)	(X4, Y4)	(Y3, Y4)
Change Rate	49.6061	49.6458	49.6237	49.6290

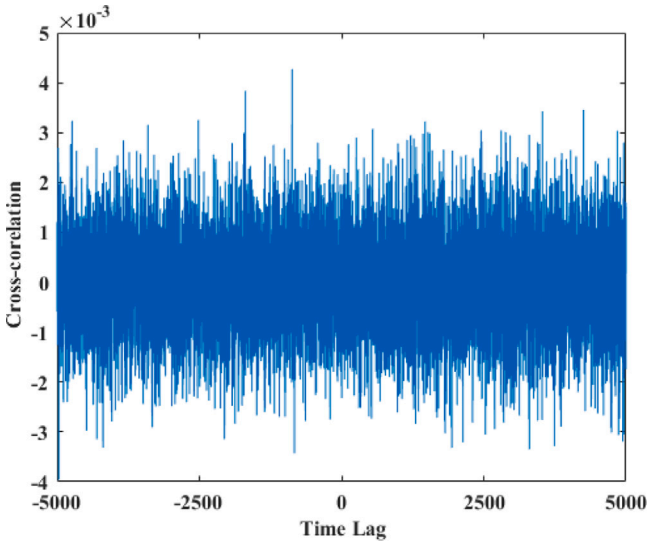


Fig. 5. Cross-correlation analysis of $\{x_t\}$ and $\{y_t\}$.

4.3. Key sensitivity analysis

Key sensitivity is an important performance indicator to qualify the PRNG. In an ideal PRNG sequence, a small change in initial value or control parameters leads to a 50% bit change rate. Since our proposed design is based on integer arithmetic, we generated the sequences with a single bit change to perform key sensitivity analysis. We generated two sequences with same initial values $\mathbf{x} = (1, 1, \ldots, 1)$, and initial values of Y_{in} as 2 and 3, corresponding to output sequences as X3, Y3 and X4, Y4, respectively. The rate of bit change between two-bit sequences S_1 and S_2 of l samples is computed as follows:

$$B = \frac{\sum_{i=1}^{l} |S_{1i} - S_{2i}|}{l} \times 100\% \tag{9}$$

From Table 4, we can observe that the generated PRNG has close to 50% bit change rate, which is an ideal case.

4.4. Cross-correlation analysis

Cross-correlation allows to compare two signals $x(t)$ and $y(t)$. It is expressed as

$$C_{xy} = \frac{\sum_{i=0}^{N} (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sqrt{\sum_{i=0}^{N} (x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=0}^{N} (y_i - \bar{y})^2}}, \tag{10}$$

where $C_{xx} \in [-1, 1]$, $\bar{x}$ and $\bar{y}$ are mean values of x_t and y_t, respectively. Fig. 5 shows C_{xy} for random sequences generated from the proposed system with $\mathbf{x} = (0, 0, 0 \ldots, 1)$ and $y_0 = 1$. This figure suggests that the cross-correlation between the two sequences is close to zero. We also evaluated the cross-correlation of sequences generated with different initial conditions and found similar results. Such results confirm the underlying sequences are uncorrelated, thereby confirming the independence of the initial conditions.

4.5. Auto-correlation analysis

Auto-correlation is useful for detecting periodicity in time series. A correlation coefficient close to 0 implies that values in the underlying sequence are different and independent. The auto-correlation of

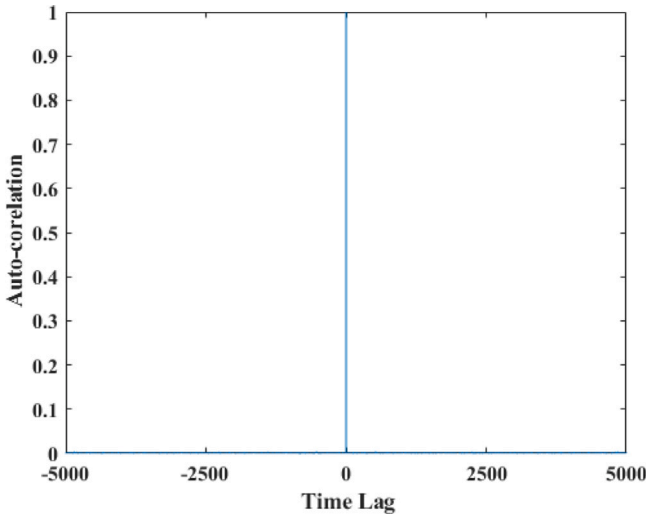


Fig. 6. Auto-correlation analysis of $\{x_t\}$.

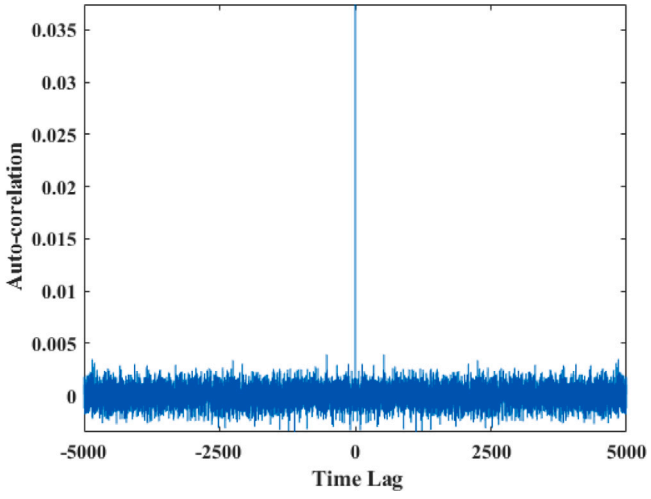


Fig. 7. Zoom-in of the auto-correlation analysis of $\{x_t\}$.

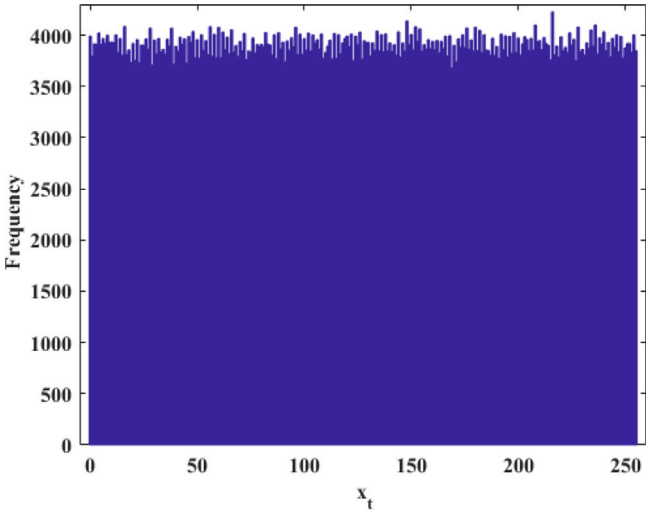


Fig. 8. Distribution of values in the random sequence $\{x_t\}$.

Table 5
Comparison of NIST 800-22 test results with other PRNG.

Sub-Tests	Proposed-$\{x_t\}$		Proposed-$\{y_t\}$		[21]		[22]		[23]	
	P-value	Proportion	P-value	Proportion	P-value	Proportion	P-value	Proportion	P-value	Proportion
1. Frequency	0.3838	98/100	0.7792	99/100	0.1370	97/100	0.8978	99/100	0.7399	99/100
2. Block Frequency	0.7598	100/100	0.4012	100/100	0.9880	98/100	0.1719	99/100	0.4372	99/100
3. Cumulative Sums	0.3896	99/100	0.7093	99/100	0.5190	99/100	0.4750	99/100	0.6579	99/100
4. Runs	0.2133	99/100	0.0457	100/100	0.2900	100/100	0.6580	100/100	0.3190	99/100
5. Longest Run	0.3669	100/100	0.8514	99/100	0.4370	100/100	0.0757	100/100	0.1087	100/100
6. Rank	0.3669	99/100	0.6787	99/100	0.5140	98/100	0.8514	100/100	0.3345	100/100
7. FFT	0.2897	97/100	0.9879	98/100	0.8980	100/100	0.4373	100/100	0.5749	98/100
8. Non Overlapping	0.7197	99/100	0.6579	99/100	0.5540	99/100	0.9879	100/100	0.5341	100/100
9. Overlapping	0.2493	97/100	0.4559	100/100	0.3840	96/100	0.2248	96/100	0.9914	99/100
10. Universal	0.4944	99/100	0.4373	100/100	0.2490	100/100	0.4373	97/100	0.4826	98/100
11. Approximate Entropy	0.2133	99/100	0.1453	97/100	0.7790	100/100	0.8677	99/100	0.0480	98/100
12. Random Excursions	0.5981	82/83	0.6718	67/68	0.3080	98/100	0.8195	61/61	0.6579	100/100
13. Random Excursions Variant	0.6761	83/83	0.5681	67/68	0.3170	98/100	0.9411	61/61	0.3838	98/100
14. Serial	0.2205	99/100	0.0179	97/100	0.7770	100/100	0.7399	99/100	0.1626	99/100
15. Linear Complexity	0.4373	98/100	0.2493	100/100	0.2130	99/100	0.2023	99/100	0.2757	100/100

Table 6
Comparative study of the proposed design with state of the art PRNGs.

Ref	Type of RN	Entropy source	ASIC/FPGA	Resources	Post Processing	No. of bits	Max. Frequency MHz	Throughput Mbits/s	Test Suite
[4]	PRNG	Bernoulli shift Map	Spartan 3E	LUT's=575 Slice Reg.'s=423 DSP's=9	XOR	52	36.90	7.380	NIST
[4]	PRNG	Tent Map	Spartan 3E	LUT's=769 Slice Reg.'s=510 DSP's=14	XOR	53	33.26	6.652	NIST
[24]	PRNG	Logistic Map	Virtex 7	LUT's=510 Slice Reg.'s=263 DSP's=13	LFSR or glibc LCG	1	132	132	NIST
[25]	TRNG	Ring Oscillator	Spartan 3A	LUT's=528 Slice Reg.'s=447	Von Neumann corrector	–	24	6	NIST
[26]	PUF-CPRNG	PUF-Chaos	Virtex-7	LUT's=631 Slice Reg.'s=1014	Physical unclonable function	16	52	832	NIST ENT Correlation
[27]	PRNG	modified Lorenz	Virtex-5	LUT's=178 Slice Reg.'s=65	–	24	300	7200	NIST
[22]	8-bit RNG	Piecewise Linear	Zynq 7000	LUT's=56 LUTRAM's=23 Slice Reg.'s=196	XOR	8	141	1136	NIST Correlation
[28]	PRNG	Four-Wing Memristive Hyperchaotic System and Bernoulli Map	Zynq XC7Z020	LUT's=24,836 Slice Reg.'s=27,371	XOR	–	135.04	62.5	NIST ENT AIS.31
[29]	PRNG	Hybrid chaotic map	ASIC(TSMC 180 *nm*)	Gate count=15,732K	LFSR	32	125	3.2	NIST Correlation
[30]	PRNG	Skew Tent map	ASIC(TSMC 180 *nm*)	Gate count=19.6K	LFSR	1	125	1000	NIST Correlation
[17]	PRNG	PWLCM	Zynq 7020	LUT's=16 Slice Reg.'s=12 –	–	8	134	1072	NIST Correlation
Proposed design	PRNG	Delayed QACM	Zynq 7000	LUT's=65 LUTRAM's=31 Slice Reg.'s=290	LFSR	16	138	2208	NIST Correlation
Proposed design	PRNG	Delayed QACM	ASIC (TSMC 90 nm)	Gate count=2.73K	LFSR	16	342	5460	NIST Correlation

a random sequence is equal to 1 only at the origin (time lag equal to 0). Fig. 6 depicts the auto-correlation of $\{x_t\}$ and its bound is calculated as $\pm 4\times 10^{-3}$ [21]. Fig. 7 presents the zoom version of Fig. 6. It suggests that the generated pseudo-random sequences have lower auto-correlation, thus qualifying as random.

We further evaluated the histogram of x_t to evaluate the data distribution. Fig. 8 shows that the distribution of x_t is quite uniform, thus confirming its randomness. The same result is obtained with sequence y_t and for various initial conditions.

4.6. NIST test analysis

The proposed architecture is synthesized on Xilinx Zynq 7000 FPGA. The random bits are generated using 8-bit unsigned integer arithmetic and the randomness of output bitstreams is evaluated using NIST 800-22 suite. It statistically evaluates the randomness of the sequence of numbers by evaluating the P-value of 15 sub tests. The sequence is said to be random if the P-value of each binary sequence or bitstream is greater than the significant level $\alpha = 0.01$. Proportion values are used to assess the uniformity of P-values in the statistical test. The P-values and proportion of the NIST test result are summarized in Table 5. From this table, we can conclude that the binary sequence generated from the proposed hardware passes all the tests and has good statistical properties to be referred as a pseudo-random number generator. Table 5 also shows that the statistical properties of the proposed PRNG are similar to those of other existing PRNG of the same type. The proposed PRNG does not require further LFSR input/output perturbations to achieve satisfactory randomness.

5. Comparison with state of the art PRNGs

We further compared the hardware performance of our PRNG with existing PRNGs and tabulated the performance parameters in Table 6. From this Table, it can be observed that the Ref. [27] gives highest throughput when implemented on Xilinx Virtex-5 FPGA, that works on 298.5 MHz at 550 MHz clock frequency. However, the proposed PRNG has attained a maximum operating frequency of 138 MHz at 100 MHz clock frequency. Further, it produces 16 bits per clock cycle leading to 2.208 Gbps throughput without using any DSP slices. Hence the proposed design is a light design for FPGA platform. Similarly, the ASIC implementation results give competitive performance compared to the state of the art implementations.

In addition to the hardware performance, the keyspace of the proposed system is $2^{480} - 1$ and its period can reach $2^{480} - 1$. Such a performance is much better than the similar PRNG considered for comparison in this paper. Moreover, the theoretical investigation shows

that depending on the need, the keyspace as well as the period of the proposed PRNG can be easily increased by choosing high order primitive polynomial in the list provided in [20]. The implementation of the proposed method includes exclusively 8-bit binary integer arithmetic operations as required in modern cryptography, instead of using 32-bit fixed/floating point arithmetic that is hardware costly. The main advantage with 8-bit integer arithmetic is that our proposed PRNG can even be implemented with low-end processors.

6. Conclusions

This work proposed a high-speed 8-bit integer true periodic orbit PRNG using delayed quantized Arnold's cat map. A light architecture of the proposed map is realized on Xilinx Zynq 7000 FPGA (with exclusively LUT, LUTRAM and FF) and 90 nm CMOS technology ASIC. The design has achieved a high throughput of 2.208 Gbps 5.46 Gbps on Zynq 7000 FPGA and ASIC, respectively. The maximum operating frequencies of the FPGA and ASIC design is 138 MHz and 342 MHz, respectively. We used NIST 800-22 and other security tests to evaluate the randomness of the design. We expect to improve the designed PRNG by optimizing its architecture and the choice of δ such that for n-bit precision, the period of the system is equal to $2^{n(\delta+2)}-1$ for all the initial conditions, in order to fulfill the stream cipher security requirement under 8-bit precision constraint.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] Öztürk I, Kiliç R. Utilizing true periodic orbits in Chaos-based cryptography. Nonlinear Dynam 2021;103:2805–18.

[2] Huang C, Ding Q. Performance of finite precision on discrete Chaotic map based on a feedback shift register. Complexity 2020;4676578.

[3] Fan C, Ding Q. Analysis and resistance of dynamic degradation of digital Chaos via functional graphs. Nonlinear Dynam 2021;103(1):1081–97.

[4] de la Fraga LG, Torres-Pérez E, Tlelo-Cuautle E, Mancillas-López C. Hardware implementation of pseudo-random number generators based on Chaotic maps. Nonlinear Dynam 2017;90(3):1661–70.

[5] Rezk AA, Madian AH, Radwan AG, Soliman AM. Reconfigurable chaotic pseudo random number generator based on FPGA. AEU-Int J Electron Commun 2019;98:174–80.

[6] Elmanfaloty RA, Abou-Bakr E. Random property enhancement of a 1D chaotic PRNG with finite precision implementation. Chaos Solitons Fractals 2019;118:134–44.

[7] Blank M. Pathologies generated by round-off in dynamical systems. Physica D 1994;78:93–114.

[8] Bakiri M, Guyeux C, Couchot J-F, Marangio L, Galatolo S. A hardware and secure pseudorandom generator for constrained devices. IEEE Trans Ind Inf 2018;14:3754–65.

[9] Wang Y, Liu Z, Zhang LY, Pareschi F, Setti G, et al. From chaos to pseudo-randomness: A case study on the 2-D coupled map lattice. IEEE Trans Cybern 2021;1–11. http://dx.doi.org/10.1109/TCYB.2021.3129808.

[10] Kumar Panda A, Chandra Ray K. A coupled variable input LCG method and its VLSI architecture for pseudorandom bit generation. IEEE Trans Instrum Meas 2020;69(4):1011–9.

[11] Öztürk I, Kiliç R. Digitally generating true orbits of binary shift chaotic maps and their conjugates. Commun Nonlinear Sci Numer Simul 2018;62:395–408.

[12] Gnyamsi GG, Djeugoue H, Eyebe Fouda JSA, Sabat SL, Koepf W. An extendable key space integer image-Cipher using 4-bit piece-wise linear cat map. Multimedia Tools Appl 2022. http://dx.doi.org/10.1007/s11042-022-13779-y.

[13] Keating JP, Mezzadri F. Pseudo-symmetries of Anosov map and spectral statistics. Nonlinearity 2000;13:747–75.

[14] Chen F, Wong K-W, Liao X, Xiang T. Period distribution of generalized discrete Arnold cat map. Theoret Comput Sci 2014;552:13–25.

[15] Souza CEC, Shavez DPB, Pimentel C. One-dimensional pseudo-chaotic sequences based on the discrete Arnold's cat map over $\mathbb{Z}_{3^m}$. IEEE Trans Circuits Syst I Regul Pap 2018;65(1):235–46. http://dx.doi.org/10.1109/TCSI.2017.2717943.

[16] Eyebe Fouda JSA, Koepf W. An 8-bit precision Cipher for fast image encryption. Multimedia Tools Appl 2022;81:34027–46.

[17] Djeugoue H, Gnyamsi GG, Eyebe Fouda JSA, Koepf W. On the implementation of large period piece-wise linear Arnold cat map. Multimedia Tools Appl 2022;81:39003–20.

[18] Gong G, Helleseth T, Kumar PV. Solomon W. Golomb—Mathematician, engineer, and Pioneer. IEEE Trans Inform Theory 2018;64:2844–57.

[19] Gardner D, Sălăgean A, Phan RCW. Efficient generation of elementary sequences. In: Stam M, editor. Cryptography and coding. Berlin, Heidelberg: Springer Berlin Heidelberg; 2013, p. 16–27.

[20] Živković M. Table of primitive binary polynomials II. Math Comp 1994;63:301–6.

[21] Yu F, Li L, He B, Liu L, Qian S, Huang Y, et al. Design and FPGA implementation of a pseudorandom number generator based on a four-wing memristive hyperchaotic system and Bernoulli map. IEEE Access 2019;7:181 884–98.

[22] Kopparthi VR, Kali A, Sabat SL, Anumandla KK, Peesapati R, et al. Hardware architecture of a digital piecewise linear chaotic map with perturbation for pseudorandom number generation. AEU - Int J Electron Commun 2022;147:154138.

[23] Gupta MD, Chauhan RK. Secure image encryption scheme using 4D-hyperchaotic systems based reconfigurable pseudo-random number generator and S-box. Integration 2021;81:137–59.

[24] Garcia-Bosque M, Pérez-Resa A, Sánchez-Azqueta C, Aldea C, Celma S. Chaos-based bitwise dynamical pseudorandom number generator on FPGA. IEEE Trans Instrum Meas 2018;68(1):291–3.

[25] Anandakumar NN, Sanadhya SK, Hashmi MS. FPGA-based true random number generation using programmable delays in oscillator-rings. IEEE Trans Circuits Syst II 2019;67(3):570–4.

[26] Kalanadhabhatta S, Kumar D, Anumandla KK, Reddy SA, Acharyya A. PUF-based secure chaotic random number generator design methodology. IEEE Trans Very Large Scale Integr (VLSI) Syst 2020;28(7):1740–4.

[27] Rezk AA, Madian AH, Radwan AG, Soliman AM. Multiplierless chaotic pseudo-random number generators. AEU-Int J Electron Commun 2020;113:152947.

[28] Yu F, Li L, He B, Liu L, Qian S, et al. Design and FPGA implementation of a pseudorandom number generator based on a four-wing memristive hyperchaotic system and Bernoulli map. IEEE Access 2019;7:181884–98.

[29] Chen S-L, Hwang T, Chang S-M, Lin W-W. A fast digital chaotic generator for secure communication. Int J Bifurcation Chaos 2010;20(12):3969–87.

[30] Garcia-Bosque M, Díez-Señorans G, Pérez-Resa A, Sánchez-Azqueta C, Aldea C, et al. A 1 Gbps Chaos-based stream Cipher implemented in 0.18 μm CMOS technology. Electronics 2019;8(6):623.